

APRENDENDO BANCO DE DADOS COM SUPABASE

Estudo dirigido

Projeto prático: banco de dados compartilhado de uma cafeteria

Ambiente

Linux, navegador atualizado, conta no Supabase, VS Code e Node.js LTS. A maior parte da aula ocorre no Dashboard e no SQL Editor do Supabase.

Versão do material: 3 de agosto de 2026

1. Apresentação do estudo dirigido

Este tutorial utiliza o sistema +Café como contexto para ensinar os fundamentos de bancos de dados relacionais no Supabase. O foco principal não é construir todas as telas do aplicativo, mas compreender como os dados são modelados, armazenados, consultados, protegidos e compartilhados entre diferentes dispositivos.

Ideia central

O Supabase entrega um banco PostgreSQL completo para cada projeto. A partir do esquema do banco, a plataforma também disponibiliza APIs, autenticação, políticas de segurança e recursos de tempo real.

1.1 O que o aluno aprenderá

- Diferenciar banco de dados, tabela, coluna, linha e tipo de dado.
- Compreender chave primária, chave estrangeira, restrições e relacionamentos.
- Criar tabelas pelo SQL Editor e visualizar dados pelo Table Editor.
- Executar operações CRUD: inserir, consultar, atualizar e excluir.
- Usar filtros, ordenação, JOIN, agrupamento, funções de agregação, view e índice.
- Compreender a diferença entre autenticação e autorização.
- Ativar Row Level Security e criar políticas básicas.
- Consultar o banco com supabase-js e observar mudanças com Realtime.

1.2 Organização das aulas

Tempo	Tema	Entrega esperada
1ª	Fundamentos e primeiro projeto	Projeto criado, primeira tabela e registros no Table Editor.
2ª	Modelagem relacional	DER compreendido e esquema +Café criado no SQL Editor.
3ª	SQL e análise dos dados	CRUD, JOIN, agrupamentos, view e índice praticados.
4ª	Auth, RLS, API e Realtime	Usuário autenticado consulta somente sua cafeteria e recebe mudanças.

1.3 Papel do monitor

O monitor deve executar cada bloco primeiro em seu próprio projeto, projetar a tela e só então pedir que os alunos repitam. A turma não deve avançar enquanto a maioria não atingir o checkpoint da iteração. Em caso de erro, peça ao aluno que leia a primeira mensagem apresentada pelo Supabase antes de alterar vários comandos ao mesmo tempo.

Regra pedagógica

Digite e execute trechos pequenos. Depois de cada comando, observe o efeito no Table Editor. Isso ajuda o aluno a relacionar o código SQL com a mudança concreta no banco.

2. Preparação do ambiente

2.1 Materiais necessários

- Computador com Linux e acesso à internet.
- Navegador Firefox, Chromium ou Google Chrome.
- Conta de e-mail para criar uma conta no Supabase.
- Visual Studio Code.
- Node.js em versão LTS e npm, usados somente na última iteração.

2.2 Verificar Node.js e npm

No terminal integrado do VS Code:

```
node --version  
npm --version
```

Os dois comandos devem apresentar números de versão. Caso node não seja reconhecido, instale uma versão LTS antes da quarta iteração. As três primeiras partes do tutorial podem ser realizadas apenas pelo navegador.

2.3 Criar uma pasta para os arquivos da aula

```
mkdir -p ~/aulas/mais-cafe-supabase  
cd ~/aulas/mais-cafe-supabase  
code .
```

No VS Code, crie os arquivos 001_schema.sql, 002_seed.sql, 003_rls.sql e 004_queries.sql. Eles também estão disponíveis no pacote prático entregue junto com este tutorial.

3. Iteração 1 - Fundamentos e primeiro projeto

Objetivo da primeira hora

Criar um projeto no Supabase, reconhecer as áreas principais do Dashboard e construir uma primeira tabela para compreender linhas, colunas e tipos.

3.1 O que é um banco de dados?

Um banco de dados é uma estrutura organizada para guardar informações e permitir que elas sejam pesquisadas e alteradas. No +Café, o banco guarda produtos, categorias, mesas, comandas e itens de cada pedido. O Supabase utiliza PostgreSQL, um sistema gerenciador de banco de dados relacional.

Conceito	Explicação simples	Exemplo no +Café
Tabela	Agrupa registros do mesmo assunto.	products guarda os produtos.
Coluna	Define uma característica do registro.	name, price e active.
Linha	Representa uma ocorrência completa.	Uma linha pode ser o produto Espresso.
Tipo de dado	Determina que tipo de valor a coluna aceita.	text, numeric, boolean e timestampz.
Esquema	Organiza objetos do banco em um espaço lógico.	As tabelas do app ficam em public.

Iteração 1 - Primeiro contato com o Supabase

SUPABASE

Table Editor

SQL Editor

Database

Authentication

Realtime

API Docs

Projeto: maiscafe-aula

Tabela products

id	name	price	active
a1...	Espresso	7.00	true
b2...	Cappuccino	12.90	true
c3...	Brownie	9.50	true

Conceitos observados

- Tabela:** conjunto organizado de registros
- Coluna:** característica do dado
- Linha:** um registro completo
- Tipo:** regra do valor armazenado

Tela didática da primeira iteração: Table Editor e conceitos básicos.

3.2 Criar o projeto no Supabase

1. Acesse o Dashboard do Supabase e entre em sua conta.
2. Crie um novo projeto dentro de uma organização.
3. Use o nome maiscafe-aula seguido de suas iniciais, por exemplo: maiscafe-aula-gj.
4. Defina uma senha forte para o banco e guarde-a em local seguro.
5. Escolha uma região próxima da turma e aguarde o projeto ficar disponível.

Estudo dirigido de 4 horas • Material para iniciantes

Atenção

A senha do banco não é a mesma coisa que a senha dos usuários do aplicativo. Ela é usada para conexões administrativas. Não publique essa senha no GitHub.

3.3 Conhecer o Dashboard

Área	Para que serve
Table Editor	Criar tabelas visualmente e editar linhas.
SQL Editor	Executar comandos SQL, salvar consultas e observar resultados.
Database	Ver tabelas, funções, índices, extensões, backups e desempenho.
Authentication	Criar e gerenciar usuários do aplicativo.
Realtime	Configurar e observar recursos de atualização em tempo real.
API Docs	Consultar a API gerada automaticamente a partir das tabelas.

3.4 Criar uma tabela experimental pelo Table Editor

Antes de escrever SQL, criaremos uma tabela visualmente. Isso facilita a associação entre os campos da interface e os elementos de uma tabela relacional.

- Abra Table Editor e escolha criar uma nova tabela.
- Nomeie a tabela como `classroom_notes`.
- Mantenha a coluna `id` criada automaticamente como chave primária.
- Adicione `title` do tipo `text` e marque como obrigatório.
- Adicione `completed` do tipo `bool` com valor padrão `false`.
- Adicione `created_at` do tipo `timestamptz` com valor padrão `now()`.
- Salve a tabela e insira duas linhas pelo próprio Table Editor.

3.5 Repetir a consulta no SQL Editor

Execute:

```
select id, title, completed, created_at
from public.classroom_notes
order by created_at desc;
```

O comando `SELECT` lê dados. A lista depois de `SELECT` define as colunas exibidas. `FROM` informa a tabela. `ORDER BY` organiza o resultado. A palavra `desc` coloca os registros mais recentes primeiro.

Atividade 1 - Reconhecendo a tabela

Na tabela `classroom_notes`, indique qual coluna identifica cada linha, qual coluna aceita verdadeiro/falso e qual coluna registra data e horário.

Resposta do aluno:

Atividade 2 - Criando um registro

Insira pelo Table Editor uma terceira linha com o título Revisar chave primária e marque `completed` como verdadeiro.

Resposta do aluno:

Checkpoint da iteração

Antes de continuar, o aluno deve confirmar:

- ☐ O projeto foi criado e está ativo.
- ☐ O aluno consegue localizar Table Editor e SQL Editor.
- ☐ A tabela `classroom_notes` possui pelo menos três linhas.
- ☐ O `SELECT` apresenta os registros sem erro.

4. Iteração 2 - Modelagem relacional do +Café

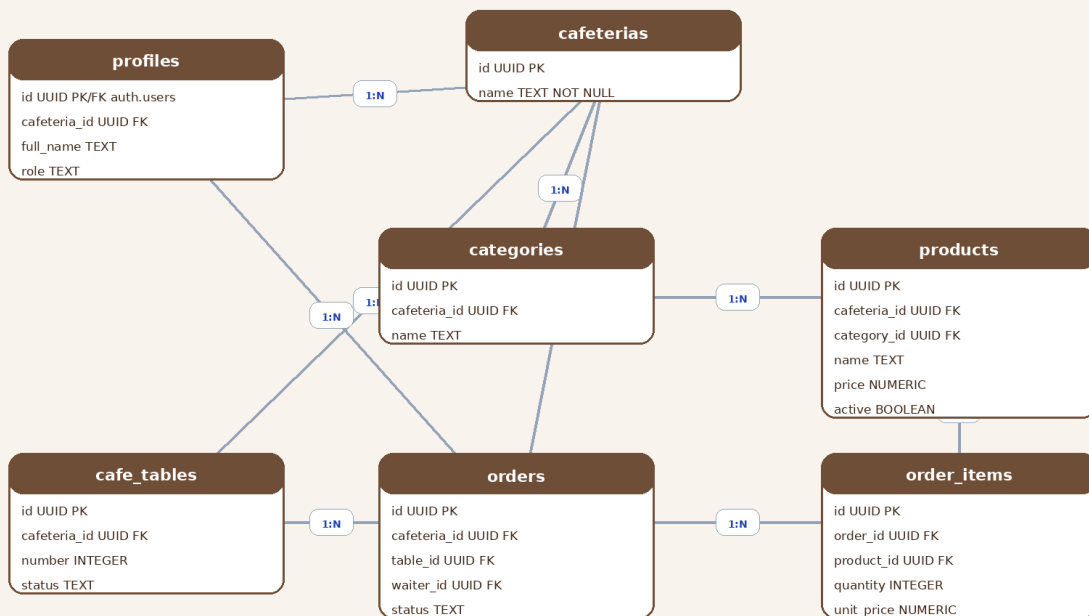
Objetivo da segunda hora

Compreender entidades, atributos, relacionamentos, chave primária, chave estrangeira e restrições; depois criar o esquema do +Café.

4.1 Modelar antes de programar

Modelagem é o processo de decidir quais informações serão armazenadas e como elas se relacionam. Um modelo bem planejado reduz duplicações, evita dados contraditórios e facilita as consultas. No +Café, não devemos repetir o nome da categoria em todas as linhas de produtos. Em vez disso, categories guarda cada categoria uma vez e products aponta para ela por meio de uma chave estrangeira.

Iteração 2 - Diagrama Entidade-Relacionamento do +Café



PK identifica cada linha • FK conecta tabelas • 1:N significa um para muitos

Exemplo: uma categoria possui muitos produtos; cada produto pertence a uma categoria.

DER simplificado utilizado na aula.

4.2 Entidade, atributo e relacionamento

Termo	Definição	Exemplo
Entidade	Objeto ou conceito sobre o qual guardamos dados.	Produto, categoria, mesa e comanda.
Atributo	Característica de uma entidade.	O preço é atributo de produto.
Relacionamento	Ligação lógica entre entidades.	Uma categoria possui muitos produtos.
Cardinalidade 1:N	Uma linha do lado 1 pode se ligar a várias do lado N.	Uma mesa pode ter várias comandas ao longo do tempo.

4.3 Chaves e integridade

A chave primária, ou PK, identifica uma linha de maneira única. A chave estrangeira, ou FK, referencia a chave primária de outra tabela. O PostgreSQL impede que uma FK aponte para um registro inexistente, preservando a integridade referencial.

Restrição	Função	Exemplo
PRIMARY KEY	Identifica unicamente cada linha.	products.id
FOREIGN KEY	Cria uma referência entre tabelas.	products.category_id → categories.id
NOT NULL	Impede ausência de valor.	products.name
UNIQUE	Impede repetição indevida.	Número da mesa dentro da cafeteria.
CHECK	Valida uma condição.	price >= 0 e quantity > 0.
DEFAULT	Define um valor automático.	active começa como true.

4.4 Por que usar UUID?

O tutorial utiliza UUID como identificador. UUIDs são úteis quando diferentes dispositivos ou serviços criam registros, pois diminuem a chance de colisão de identificadores. Para valores de dinheiro, usamos numeric(10,2), evitando cálculos financeiros com números de ponto flutuante.

Boa prática

Evite armazenar preço apenas como texto. Ao usar numeric, o banco consegue comparar, somar e calcular médias corretamente.

4.5 Criar o esquema

Abra o arquivo 001_schema.sql no VS Code, copie o conteúdo e execute no SQL Editor. Para a aula, o monitor pode executar a primeira tabela, observar o Table Editor e só depois executar o restante.

Exemplo: tabela categories

```
create table public.categories (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  name text not null,
  created_at timestamptz not null default now(),
  unique (cafeteria_id, name)
);
```

A cláusula on delete cascade significa que, se uma cafeteria for removida, suas categorias também serão removidas. Isso é adequado neste exemplo didático, mas exclusões em produção devem ser cuidadosamente planejadas.

4.6 Inserir dados iniciais

Execute 002_seed.sql. O script cria uma cafeteria, categorias, produtos e oito mesas. Em seguida, abra Table Editor e confira as tabelas categories, products e cafe_tables.

Primeira consulta com relacionamento:

```
select c.name as categoria, p.name as produto, p.price
from public.products p
join public.categories c on c.id = p.category_id
order by c.name, p.name;
```


Atividade 3 - Lendo o DER

Explique com suas palavras o relacionamento entre categories e products e indique onde fica a chave estrangeira.

Resposta do aluno:

Atividade 4 - Criando uma restrição

Escreva uma coluna stock_quantity do tipo integer que não aceite valores negativos e que comece com zero.

Resposta do aluno:

Atividade 5 - Criando uma nova categoria

Insira a categoria Combos para a cafeteria da aula e confirme o novo registro no Table Editor.

Resposta do aluno:

Checkpoint da iteração

Antes de continuar, o aluno deve confirmar:

- ☐ As sete tabelas principais aparecem no Table Editor.
- ☐ Os dados de exemplo foram inseridos.
- ☐ O aluno consegue apontar PK e FK no DER.
- ☐ A consulta com JOIN apresenta categoria, produto e preço.

5. Iteração 3 - CRUD, consultas e análise dos dados

Objetivo da terceira hora

Manipular dados com SQL e compreender filtros, ordenação, JOIN, agregações, views e índices.

5.1 O que significa CRUD?

Letra	Operação	Comando SQL	Exemplo
C	Create	INSERT	Cadastrar um produto.
R	Read	SELECT	Listar os produtos.
U	Update	UPDATE	Alterar o preço.
D	Delete	DELETE	Excluir um registro de teste.

5.2 Inserir um produto

```
insert into public.products (cafeteria_id, category_id, name, description, price)
select
  f.id,
  c.id,
  'Mocha',
  'Café com leite e chocolate',
  14.50
from public.cafeterias f
join public.categories c on c.cafeteria_id = f.id
where f.name = '+Café - Unidade Escola'
and c.name = 'Cafés';
```

O INSERT cria uma linha. Neste exemplo, SELECT encontra os identificadores da cafeteria e da categoria, evitando copiar UUIDs manualmente.

5.3 Consultar, filtrar e ordenar

```
select name, price
from public.products
where active = true
and price <= 10
order by price asc, name asc;
```

WHERE filtra linhas. AND exige que as duas condições sejam verdadeiras. ORDER BY organiza primeiro pelo menor preço e depois pelo nome.

5.4 Atualizar com segurança

```
update public.products
set price = 15.00
where name = 'Mocha'
returning id, name, price;
```

Cuidado com UPDATE e DELETE

Sempre confira o WHERE antes de executar. Sem WHERE, o comando pode alterar ou remover todas as linhas da tabela. Use primeiro um SELECT com a mesma condição.

5.5 Excluir um registro de teste

```
select id, name from public.products where name = 'Mocha';

delete from public.products
where name = 'Mocha'
returning id, name;
```

RETURNING mostra as linhas afetadas. Ele ajuda a confirmar o resultado da operação.

5.6 JOIN e funções de agregação

Iteração 3 - Consultas SQL, JOIN e agrupamentos

SQL Editor

```
select
  c.name as categoria,
  count(p.id) as quantidade,
  round(avg(p.price), 2) as preco_medio
from categories c
left join products p on p.category_id = c.id
where p.active = true
group by c.name
```

order by preco_medio desc;

Resultado

categoria	quantidade	preco_medio
Cafés	3	10.63
Doces	2	9.25
Bebidas	2	5.50

O que ocorreu?

- JOIN conectou
- GROUP BY reuniu
- COUNT contou
- AVG calculou média

Consulta que conecta tabelas e produz um resumo por categoria.

JOIN combina linhas relacionadas. COUNT conta registros. AVG calcula a média. GROUP BY reúne linhas que possuem a mesma categoria. Essas operações permitem gerar relatórios sem duplicar os dados.

```
select
  c.name as categoria,
  count(p.id) as quantidade,
  round(avg(p.price), 2) as preco_medio
from public.categories c
left join public.products p
  on p.category_id = c.id and p.active = true
group by c.id, c.name
order by c.name;
```

5.7 Views

Uma view é uma consulta salva com um nome. O esquema cria v_order_totals para calcular o total de cada comanda. A view não duplica os registros; ela apresenta o resultado da consulta sempre que for acessada.

```
select * from public.v_order_totals;
```

5.8 Índices

Um índice ajuda o PostgreSQL a localizar linhas mais rapidamente, especialmente quando as tabelas crescem. O custo é ocupar espaço e tornar inserções e atualizações um pouco mais trabalhosas. Por isso, não se cria índice em toda coluna sem analisar as consultas.

```
create index idx_products_cafeteria_active  
on public.products(cafeteria_id, active);
```

Atividade 6 - Filtrando produtos

Escreva uma consulta que mostre nome e preço dos produtos ativos com preço entre R\$ 8,00 e R\$ 13,00, do maior para o menor.

Resposta do aluno:

Atividade 7 - Atualizando uma mesa

Altere o status da Mesa 3 para reservada e use RETURNING para confirmar.

Resposta do aluno:

Atividade 8 - Relatório por categoria

Escreva uma consulta que mostre a categoria e o menor preço de produto em cada categoria.

Resposta do aluno:

Atividade 9 - Entendendo índices

Explique por que criar um índice em orders(table_id, status) pode ajudar a tela de uma mesa.

Resposta do aluno:

Checkpoint da iteração

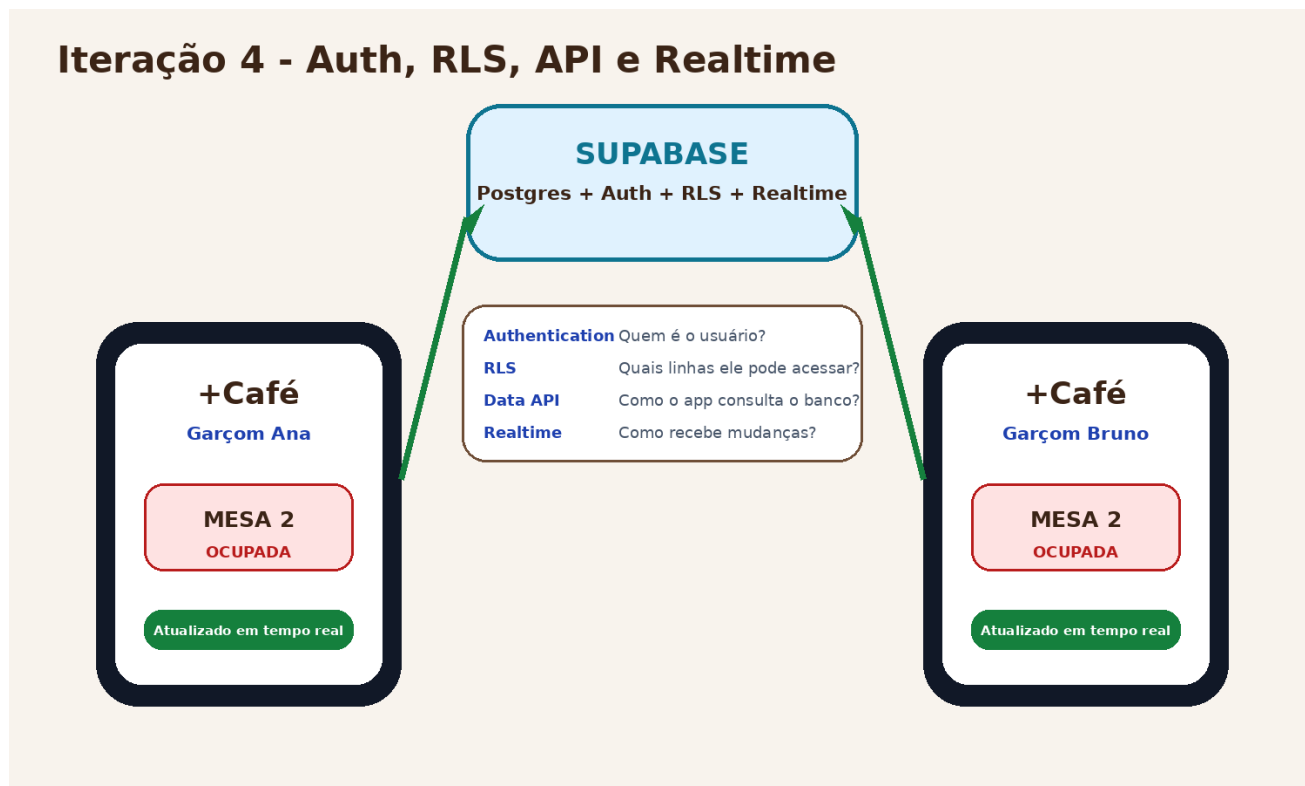
Antes de continuar, o aluno deve confirmar:

- ☐ O aluno executou INSERT, SELECT, UPDATE e DELETE.
- ☐ Conseguiu explicar a função de WHERE e o risco de omiti-lo.
- ☐ Executou um JOIN e uma consulta com GROUP BY.
- ☐ Entendeu que view salva uma consulta e índice ajuda buscas frequentes.

6. Iteração 4 - Auth, RLS, API e Realtime

Objetivo da quarta hora

Compreender como o Supabase identifica usuários, autoriza o acesso às linhas, gera uma API a partir do esquema e distribui mudanças entre dispositivos.



Dois aplicativos de garçons utilizando a mesma base central.

6.1 Autenticação e autorização não são a mesma coisa

Conceito	Pergunta respondida	No +Café
Autenticação	Quem é o usuário?	O Supabase Auth confirma e-mail e senha do garçom.
Autorização	O que ele pode acessar?	RLS limita o garçom aos dados da própria cafeteria.

O Supabase Auth armazena identidades em um esquema especial chamado auth. A tabela public.profiles complementa os usuários com nome, papel e cafeteria. O id do perfil referencia auth.users.id.

6.2 Criar um usuário para a aula

- Abra Authentication e acesse a lista de usuários.
- Crie um usuário com e-mail de teste e senha temporária forte.
- No SQL Editor, execute `select id, email from auth.users;` e copie o UUID do usuário.
- Encontre o UUID da cafeteria da aula.
- Insira o perfil relacionando os dois identificadores.

Substitua os UUIDs antes de executar:

```
insert into public.profiles (id, cafeteria_id, full_name, role)
values (
  'UUID_DO_USUARIO',
  'UUID_DA_CAFETERIA',
```

```
'Garçon da Aula',
'garcom'
);
```

6.3 Row Level Security

RLS adiciona condições de segurança às consultas. Uma política funciona como um filtro obrigatório aplicado pelo PostgreSQL. Tabelas do esquema public que serão acessadas pelo aplicativo devem ter RLS e políticas adequadas.

```
alter table public.products enable row level security;

create policy "members manage products"
on public.products
for all
to authenticated
using (
  cafeteria_id = (
    select p.cafeteria_id
    from public.profiles p
    where p.id = auth.uid()
  )
)
with check (
  cafeteria_id = (
    select p.cafeteria_id
    from public.profiles p
    where p.id = auth.uid()
  )
);
```

USING controla quais linhas podem ser lidas, alteradas ou excluídas. WITH CHECK controla quais dados podem ser inseridos ou gravados. auth.uid() representa o usuário autenticado da requisição.

Chaves do projeto

A chave publicável pode ser usada pelo cliente quando RLS está corretamente configurado. A service_role possui privilégios elevados e nunca deve ser colocada no aplicativo, em arquivos públicos ou no GitHub.

6.4 A API gerada automaticamente

O Supabase gera uma Data API diretamente do esquema do PostgreSQL. Quando uma tabela ou relacionamento é criado, a API passa a refletir essa estrutura. As requisições realizadas pelo cliente carregam o token do usuário e são avaliadas pelas políticas RLS.

6.5 Preparar um cliente no VS Code

No terminal:

```
mkdir -p ~/aulas/mais-cafe-supabase/cliente
cd ~/aulas/mais-cafe-supabase/cliente
npm init -y
npm install @supabase/supabase-js dotenv
cp ../arquivos_praticos/client.mjs .
cp ../arquivos_praticos/.env.example .env
```

Abra .env e preencha a URL, a chave publicável, o e-mail e a senha do usuário da aula. O arquivo .env deve ser adicionado ao .gitignore em projetos reais.

Trecho principal do cliente:

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_PUBLISHABLE_KEY
)

const { error } = await supabase.auth.signInWithPassword({
  email: process.env.AULA_EMAIL,
  password: process.env.AULA_PASSWORD,
})
```

Depois do login, a consulta `from(products).select(...)` será enviada com o token da sessão. O banco aplicará a política RLS antes de retornar os dados.

6.6 Testar o cliente

```
node client.mjs
```

O terminal deve mostrar os produtos autorizados e, depois, informar que está escutando mudanças em `cafe_tables`.

6.7 Habilitar e observar Realtime

```
alter publication supabase_realtime
add table public.cafe_tables;
```

Mantenha `client.mjs` aberto. No Table Editor, altere o status de uma mesa. O terminal deverá receber um evento. Para fins didáticos, usamos Postgres Changes por exigir menos configuração. Em sistemas maiores, a documentação recomenda avaliar Broadcast por escalabilidade e segurança.

Atividade 10 - Autenticação versus autorização

Explique por que um usuário pode estar autenticado e, mesmo assim, receber uma lista vazia ao consultar `products`.

Resposta do aluno:

Atividade 11 - Criando uma política conceitual

Descreva, sem necessidade de SQL completo, uma regra para permitir que o garçom leia somente mesas da própria cafeteria.

Resposta do aluno:

Atividade 12 - Tempo real

Com o listener aberto, altere a Mesa 4 para limpando e registre no espaço abaixo qual evento apareceu no terminal.

Resposta do aluno:

Checkpoint da iteração

Antes de continuar, o aluno deve confirmar:

- ☐ O usuário existe em Auth e possui uma linha correspondente em profiles.
- ☐ RLS está habilitado e as políticas foram executadas.
- ☐ O cliente Node faz login e consulta `products`.
- ☐ Uma alteração em `cafe_tables` aparece no listener Realtime.

7. Revisão geral

Conceito	Pergunta de revisão
Tabela e linha	Qual é a diferença entre a estrutura da tabela e um registro?
PK e FK	Como o banco identifica uma linha e conecta duas tabelas?
CRUD	Quais comandos criam, leem, atualizam e excluem dados?
JOIN	Por que consultar produtos e categorias exige um relacionamento?
RLS	Por que não basta esconder botões na interface?
API	Como o aplicativo acessa o banco sem escrever uma API CRUD do zero?
Realtime	Como dois garçons podem observar a mesma mudança de mesa?

7.1 Desafio final

Crie a tabela `clients` com `id`, `cafeteria_id`, `name`, `phone`, `email`, `active` e `created_at`. Defina tipos, chave primária, chave estrangeira, obrigatoriedade do nome, valor padrão de `active` e uma restrição que impeça dois clientes da mesma cafeteria com o mesmo e-mail quando o e-mail estiver preenchido. Insira três clientes, liste-os em ordem alfabética e proponha uma política RLS baseada em `cafeteria_id`.

Entrega sugerida

O aluno entrega: captura do DER ou descrição do modelo; arquivo SQL; resultado da consulta; e uma explicação de até cinco linhas sobre a política de segurança.

7.2 Critérios de avaliação formativa

Critério	Evidência	Peso sugerido
Compreensão conceitual	Explica tabela, linha, PK, FK e relacionamento.	25%
Modelagem	Cria tabelas com tipos e restrições adequados.	25%
SQL	Executa CRUD, filtros e JOIN sem alterar dados indevidamente.	25%
Segurança	Diferencia Auth e RLS e propõe política coerente.	15%
Organização	Mantém scripts nomeados, comentados e executáveis.	10%

8. Erros comuns e como investigar

Mensagem ou situação	Causa provável	Como resolver
relation does not exist	Tabela não foi criada ou nome está diferente.	Confira Table Editor, schema public e ordem dos scripts.
duplicate key value violates unique constraint	Valor que deveria ser único já existe.	Localize o registro existente ou altere o valor.
violates foreign key constraint	A FK aponta para uma linha inexistente.	Confirme o UUID da tabela referenciada.
violates check constraint	O valor não atende à regra CHECK.	Revise status, preço ou quantidade.
new row violates row-level security policy	A política não permite o INSERT ou falta WITH CHECK.	Confirme login, profile, cafeteria_id e política.
Consulta no app retorna []	RLS permitiu zero linhas ou usuário está em outra cafeteria.	Teste o perfil e a condição da política.

Invalid API key	Chave ou URL incorreta.	Copie a chave publicável e a URL do painel Connect/API.
Realtime não recebe eventos	Tabela não foi adicionada à publicação ou canal não assinou.	Verifique publicação, RLS e status do subscribe.

Método de depuração

- 1) Leia a mensagem completa.
- 2) Identifique o comando que falhou.
- 3) Execute um SELECT para observar o estado atual.
- 4) Corrija apenas uma hipótese.
- 5) Repita o teste.

Apêndice A - Respostas das atividades

Atividade 1

id identifica cada linha e é a chave primária. completed é boolean e aceita true ou false. created_at guarda data e horário no tipo timestampz.

Atividade 2

A terceira linha deve ter title igual a Revisar chave primária, completed igual a true e created_at preenchido automaticamente.

Atividade 3

O relacionamento é 1:N: uma categoria pode possuir muitos produtos, mas cada produto referencia uma categoria. A FK é products.category_id e aponta para categories.id.

Atividade 4

```
stock_quantity integer not null default 0
check (stock_quantity >= 0)
```

Atividade 5

```
insert into public.categories (cafeteria_id, name)
select id, 'Combos'
from public.cafeterias
where name = '+Café - Unidade Escola';
```

Atividade 6

```
select name, price
from public.products
where active = true
      and price between 8 and 13
order by price desc, name asc;
```

Atividade 7

```
update public.cafe_tables
set status = 'reservada'
where number = 3
returning number, status;
```

Atividade 8

```
select c.name as categoria, min(p.price) as menor_preco
from public.categories c
left join public.products p on p.category_id = c.id
group by c.id, c.name
order by c.name;
```

Atividade 9

A tela de uma mesa consulta frequentemente as comandas daquela mesa e filtra comandas abertas ou fechadas. Um índice composto por `table_id` e `status` pode reduzir a quantidade de linhas examinadas quando a tabela crescer.

Atividade 10

Autenticação apenas confirma a identidade. A política RLS ainda pode não encontrar um profile, encontrar uma cafeteria diferente ou negar as linhas. Por isso, um usuário válido pode receber zero registros.

Atividade 11

A regra deve comparar `cafe_tables.cafeteria_id` com a `cafeteria_id` do profile cujo id é igual a `auth.uid()`. Assim, cada usuário autenticado enxerga somente as mesas de sua cafeteria.

Atividade 12

O evento esperado é UPDATE na tabela `cafe_tables`, com os dados anteriores e/ou novos dependendo da configuração. O payload deve indicar que o status da Mesa 4 mudou para limpando.

Apêndice B - Esquema SQL completo

O conteúdo abaixo é o mesmo arquivo 001_schema.sql do pacote prático.

```
-- +Café - esquema didático para Supabase/Postgres
-- Execute no SQL Editor do Supabase.

create extension if not exists pgcrypto;

create table public.cafeterias (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  created_at timestamptz not null default now()
);

create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  full_name text not null,
  role text not null default 'garcom'
  check (role in ('admin', 'garcom', 'caixa')),
  created_at timestamptz not null default now()
);

create table public.categories (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  name text not null,
  created_at timestamptz not null default now(),
  unique (cafeteria_id, name)
);

create table public.products (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  category_id uuid not null references public.categories(id) on delete restrict,
  name text not null,
  description text,
  price numeric(10,2) not null check (price >= 0),
  active boolean not null default true,
  created_at timestamptz not null default now()
);

create table public.cafe_tables (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  number integer not null check (number > 0),
  status text not null default 'livre'
  check (status in ('livre', 'ocupada', 'reservada', 'limpando')),
  created_at timestamptz not null default now(),
  unique (cafeteria_id, number)
);

create table public.orders (
  id uuid primary key default gen_random_uuid(),
  cafeteria_id uuid not null references public.cafeterias(id) on delete cascade,
  table_id uuid not null references public.cafe_tables(id) on delete restrict,
  waiter_id uuid not null references public.profiles(id) on delete restrict,
  status text not null default 'aberta'
  check (status in ('aberta', 'fechada', 'cancelada')),
  opened_at timestamptz not null default now(),
  closed_at timestamptz
);

create table public.order_items (
  id uuid primary key default gen_random_uuid(),
  order_id uuid not null references public.orders(id) on delete cascade,
  product_id uuid not null references public.products(id) on delete restrict,
  quantity integer not null check (quantity > 0),
  unit_price numeric(10,2) not null check (unit_price >= 0),
  notes text
);

create index idx_products_category_id on public.products(category_id);
```

```
create index idx_products_cafeteria_active on public.products(cafeteria_id, active);
create index idx_orders_table_status on public.orders(table_id, status);
create index idx_order_items_order_id on public.order_items(order_id);

create or replace view public.v_order_totals
with (security_invoker = true)
as
select
  o.id as order_id,
  o.cafeteria_id,
  o.table_id,
  o.status,
  coalesce(sum(oi.quantity * oi.unit_price), 0)::numeric(10,2) as total
from public.orders o
left join public.order_items oi on oi.order_id = o.id
group by o.id, o.cafeteria_id, o.table_id, o.status;
```

Apêndice C - Dados iniciais

```
-- +Café - dados de exemplo
-- Execute depois de 001_schema.sql.

insert into public.cafeterias (name)
values ('+Café - Unidade Escola');

with cafeteria as (
  select id from public.cafeterias
  where name = '+Café - Unidade Escola'
  limit 1
)
insert into public.categories (cafeteria_id, name)
select id, category_name
from cafeteria
cross join (values ('Cafés'), ('Doces'), ('Salgados'), ('Bebidas')) as v(category_name);

with cafeteria as (
  select id from public.cafeterias where name = '+Café - Unidade Escola' limit 1
), category_ids as (
  select c.id, c.name from public.categories c
  join cafeteria f on f.id = c.cafeteria_id
)
insert into public.products (cafeteria_id, category_id, name, description, price)
select f.id, c.id, p.name, p.description, p.price
from cafeteria f
join (values
  ('Cafés', 'Espresso', 'Café curto e encorpado', 7.00::numeric),
  ('Cafés', 'Cappuccino', 'Café, leite e espuma', 12.90::numeric),
  ('Cafés', 'Café coado', 'Café filtrado tradicional', 8.00::numeric),
  ('Doces', 'Brownie', 'Brownie de chocolate', 9.50::numeric),
  ('Doces', 'Bolo de cenoura', 'Fatia com cobertura', 9.00::numeric),
  ('Salgados', 'Pão de queijo', 'Porção com 3 unidades', 8.50::numeric),
  ('Bebidas', 'Água com gás', 'Garrafa 500 ml', 5.00::numeric),
  ('Bebidas', 'Suco de laranja', 'Copo 300 ml', 9.00::numeric)
) as p(category_name, name, description, price) on true
join category_ids c on c.name = p.category_name;

with cafeteria as (
  select id from public.cafeterias where name = '+Café - Unidade Escola' limit 1
)
insert into public.cafe_tables (cafeteria_id, number, status)
select id, n, 'livre'
from cafeteria
cross join generate_series(1, 8) as n;
```

Apêndice D - Políticas RLS

```
-- +Café - segurança com Row Level Security
-- Pré-requisito: crie usuários em Authentication > Users e cadastre seus perfis.

alter table public.profiles enable row level security;
alter table public.categories enable row level security;
alter table public.products enable row level security;
alter table public.cafe_tables enable row level security;
alter table public.orders enable row level security;
alter table public.order_items enable row level security;

-- Cada usuário autenticado pode consultar o próprio perfil.
create policy "user reads own profile"
on public.profiles
for select
to authenticated
using (id = auth.uid());

-- Membros autenticados acessam somente registros da própria cafeteria.
create policy "members manage categories"
on public.categories
for all
to authenticated
using (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
)
with check (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
);

create policy "members manage products"
on public.products
for all
to authenticated
using (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
)
with check (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
);

create policy "members manage tables"
on public.cafe_tables
for all
to authenticated
using (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
)
with check (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
);

create policy "members manage orders"
on public.orders
for all
to authenticated
using (
  cafeteria_id = (
    select p.cafeteria_id from public.profiles p where p.id = auth.uid()
  )
)
with check (
```

```
cafeteria_id = (  
  select p.cafeteria_id from public.profiles p where p.id = auth.uid()  
)  
);  
  
-- Itens são autorizados pela cafeteria da comanda relacionada.  
create policy "members manage order items"  
on public.order_items  
for all  
to authenticated  
using (  
  exists (  
    select 1  
    from public.orders o  
    join public.profiles p on p.cafeteria_id = o.cafeteria_id  
    where o.id = order_items.order_id  
    and p.id = auth.uid()  
  )  
)  
with check (  
  exists (  
    select 1  
    from public.orders o  
    join public.profiles p on p.cafeteria_id = o.cafeteria_id  
    where o.id = order_items.order_id  
    and p.id = auth.uid()  
  )  
)  
);
```


Apêndice E - Cliente JavaScript

```
import 'dotenv/config'
import { createClient } from '@supabase/supabase-js'

const url = process.env.SUPABASE_URL
const key = process.env.SUPABASE_PUBLISHABLE_KEY
const email = process.env.AULA_EMAIL
const password = process.env.AULA_PASSWORD

if (!url || !key || !email || !password) {
  throw new Error('Preencha todas as variáveis do arquivo .env')
}

const supabase = createClient(url, key)

async function main() {
  const { error: loginError } = await supabase.auth.signInWithPassword({
    email,
    password,
  })

  if (loginError) throw loginError

  const { data, error } = await supabase
    .from('products')
    .select('id, name, price, active, categories(name)')
    .eq('active', true)
    .order('name')

  if (error) throw error

  console.table(data)

  const channel = supabase
    .channel('classroom-table-changes')
    .on(
      'postgres_changes',
      { event: '*', schema: 'public', table: 'cafe_tables' },
      (payload) => console.log('Mudança recebida:', payload)
    )
    .subscribe((status) => console.log('Realtime:', status))

  console.log('Escutando alterações em cafe_tables. Pressione Ctrl+C para sair.')

  process.on('SIGINT', async () => {
    await supabase.removeChannel(channel)
    process.exit(0)
  })
}

main().catch((error) => {
  console.error('Falha:', error.message)
  process.exit(1)
})
```

Referências oficiais

- **Supabase Database overview:** <https://supabase.com/docs/guides/database/overview>
- **Tables and Data:** <https://supabase.com/docs/guides/database/tables>
- **Querying Joins and Nested Tables:** <https://supabase.com/docs/guides/database/joins-and-nesting>
- **Row Level Security:** <https://supabase.com/docs/guides/database/postgres/row-level-security>
- **Managing Indexes in Postgres:** <https://supabase.com/docs/guides/database/postgres/indexes>
- **Data REST API:** <https://supabase.com/docs/guides/api>
- **Supabase Auth:** <https://supabase.com/docs/guides/auth>
- **Password-based Auth:** <https://supabase.com/docs/guides/auth/passwords>

- **Initializing supabase-js:** <https://supabase.com/docs/reference/javascript/initializing>
- **Subscribing to Database Changes:** <https://supabase.com/docs/guides/realtime/subscribing-to-database-changes>

A interface do Dashboard pode receber pequenas alterações ao longo do tempo. Quando o nome de um menu estiver diferente, procure a função equivalente na navegação atual e consulte a documentação oficial.